Lec 8

28th July, 2026

Infix, Prefix, and Postfix Notation — Complete DSA Notes

---

1. Why This Topic Exists in the First Place

Every arithmetic expression you write, like `A + B * C`, is a string of operators and operands arranged in a specific order. The order in which you *write* the operators relative to the operands is called **notation**. There are exactly three notations you need to know:

Notation	Operator Position	Example
Infix	Between the two operands	`A + B`
Prefix (Polish Notation)	Before the two operands	`+ A B`
Postfix (Reverse Polish Notation)	After the two operands	`A B +`

The problem with **infix** — the notation humans naturally use — is that a computer cannot evaluate it directly without extra rules like **operator precedence** (BODMAS/PEMDAS) and **parentheses** to resolve ambiguity. For example, `A + B * C` is ambiguous to a naive left-to-right reader: should it compute `(A+B)*C` or `A+(B*C)`? Humans resolve this using memorized precedence rules. A computer would need to build a parse tree or repeatedly scan the expression to know what to do first.

Prefix and postfix notations remove this ambiguity entirely. There is no need for precedence rules or parentheses because the *position* of the operator already tells you exactly which operands it applies to and in what order. This is why compilers, calculators, and virtual machines internally convert infix expressions into postfix (or prefix) before evaluating them — it can be evaluated in a single linear left-to-right scan using a stack, in O(n) time, with no backtracking and no need to "look ahead" to resolve precedence.

---

2. Infix Notation — The Human-Readable Form

2.1 Definition

In infix notation, every binary operator sits **between** its two operands. Example: `A + B`, `(A + B) * C`, `A + B * C - D / E`

2.2 Why It's Hard for Machines

1. **Precedence ambiguity**: `*` and `/` must be evaluated before `+` and `-` unless parentheses override this.
2. **Associativity ambiguity**: `A - B - C` must be evaluated left to right as `(A-B)-C`, not `A-(B-C)`. For exponentiation, `A ^ B ^ C` is typically right-associative: `A^(B^C)`.
3. **Parentheses**: Nested parentheses force certain sub-expressions to be evaluated first, and a machine must track opening/closing pairs correctly.
4. Because of all this, evaluating infix directly requires either:
   - Building a full **expression tree** (parse tree) and then evaluating it recursively, or
   - Using **two stacks** (one for operators, one for operands) and applying precedence rules on the fly (this is essentially the "shunting-yard" style evaluation), or
   - Converting infix → postfix/prefix first, then evaluating that with a single stack (the standard, cleanest approach).

2.3 Operator Precedence (Standard Convention Used in DSA)

From lowest to highest precedence:

Precedence	Operators	Associativity
1 (lowest)	`+` `-` (binary)	Left to Right
2	`*` `/` `%`	Left to Right
3 (highest)	`^` (exponentiation)	Right to Left
—	`( )`	Highest — forces evaluation of enclosed expression first

This precedence table is the backbone of every conversion algorithm below. Memorize it exactly — most bugs in infix-to-postfix code come from getting precedence or associativity wrong for `^`.

---

3. Prefix Notation (Polish Notation)

3.1 Definition

The operator comes **before** its operands. Example: Infix `A + B` → Prefix `+ A B` Infix `(A + B) * C` → Prefix `* + A B C`

3.2 How to Read Prefix (Mental Model)

Read prefix **right to left**. Every time you encounter an operand, push it mentally onto a stack. Every time you encounter an operator, it applies to the **top two operands you've seen so far** (pop two, combine, push result back). This right-to-left, stack-based reading is exactly how a computer evaluates it too (see Section 6).

3.3 Why Prefix Is Useful

- No parentheses ever needed — the structure is unambiguous from operator position alone.
- Easy to convert into a recursive function call structure: `+ A B` is literally `add(A, B)`, and `* + A B C` is `multiply(add(A,B), C)`. This is why prefix maps naturally onto **abstract syntax trees** and **Lisp-style languages** (Lisp literally uses prefix: `(+ A B)`).

---

4. Postfix Notation (Reverse Polish Notation, RPN)

4.1 Definition

The operator comes **after** its operands. Example: Infix `A + B` → Postfix `A B +` Infix `(A + B) * C` → Postfix `A B + C *`

4.2 How to Read Postfix (Mental Model)

Read postfix **left to right**. Every operand goes onto a stack. Every operator pops the top two operands, applies the operation, and pushes the result back. This is the single most important mental model in this entire topic — nearly every postfix algorithm (evaluation, conversion) is a direct implementation of this idea.

4.3 Why Postfix Is the Industry Favorite

- It matches how **stack machines** and **most real calculators/compilers** (e.g., old HP calculators, JVM bytecode, PostScript) actually execute instructions.
- Evaluation needs only **one stack**, one left-to-right pass, O(n) time, O(n) space — no backtracking, no recursion required (though it can be done recursively too).
- This is why, in almost all DSA courses and interviews, postfix is the "final" target form: **infix is converted to postfix, and postfix is evaluated using a stack.**

---

5. Converting Infix to Postfix (The Shunting-Yard Style Algorithm)

This is the single most important algorithm in this entire topic. Study every step carefully.

5.1 Data Structures Needed

- An **output list/string** to build the postfix expression.
- A **stack** to temporarily hold operators and parentheses.

5.2 The Full Algorithm (Step by Step)

Scan the infix expression **left to right**, one token (character) at a time:

1. **If the token is an operand** (a letter/number), append it directly to the output.
   - *Why:* operands never need reordering; their relative order is already correct in postfix.
2. **If the token is `(`**, push it onto the stack.
   - *Why:* `(` acts as a "wall" or precedence-reset marker — nothing inside it can be popped out until its matching `)` is found.
3. **If the token is `)`**, pop from the stack and append to output **repeatedly** until you pop a `(`. Discard both parentheses (don't add them to output).
   - *Why:* everything between the matching `(` and `)` must be fully resolved before anything outside the parentheses, so we flush the whole bracketed sub-expression's operators to the output now.
4. **If the token is an operator `op1`:**
   - While the stack is **not empty**, AND the top of the stack is **not `(`**, AND (precedence(top of stack) is **greater than** precedence(op1), OR (precedence(top of stack) **equals** precedence(op1) AND op1 is **left-associative**)):
     - Pop the stack and append the popped operator to the output.
   - After the while loop ends, push `op1` onto the stack.
   - *Why this condition:* An operator already on the stack should be resolved first (popped out) if it has strictly higher precedence, OR if it has equal precedence and the current operator is left-associative (because left-associative operators must be evaluated left to right, so the earlier one — sitting on the stack — must come out first). For **right-associative** operators (like `^`), equal precedence does *not* trigger a pop — the new operator is pushed on top and waits, so that the rightmost `^` gets evaluated first, correctly modeling right-to-left grouping.
5. **After the entire input is scanned**, pop all remaining operators from the stack and append them to the output.

5.3 Fully Worked Example

Convert infix `A + B * C - D` to postfix.

Step	Token	Action	Stack (top→right)	Output
1	A	operand → output	(empty)	A
2	+	stack empty → push	+	A
3	B	operand → output	+	A B
4	*	prec*() > prec(+) on stack? Top is +, prec(+) < prec()*, so no pop → push *	+ *	A B
5	C	operand → output	+ *	A B C
6	-	Top is *, prec()* > prec(-) → pop * to output. Next top is +, prec(+) == prec(-), left-assoc → pop + to output. Stack now empty → push -	-	A B C * +
7	D	operand → output	-	A B C * + D
End	—	pop remaining: -	(empty)	A B C * + D -

Final Postfix: `A B C * + D -`

Verify by evaluating both forms with values A=2, B=3, C=4, D=5: Infix: `2 + 3*4 - 5` = `2 + 12 - 5` = `9` Postfix eval: push 2, push 3, push 4 → `*` → pop 4,3 → 12, push 12 → stack: [2,12] → `+` → pop 12,2 → 14, push 14 → stack:[14] → push 5 → stack:[14,5] → `-` → pop 5,14 → 9. . Matches.

5.4 Worked Example With Parentheses

Convert `(A + B) * (C - D)` to postfix.

Step	Token	Action	Stack	Output
1	(	push	(	—
2	A	output	(	A
3	+	push (stack top is `(`, condition fails)	(+	A
4	B	output	(+	A B
5	)	pop until `(`: pop + → output; pop (→ discard	(empty)	A B +
6	*	push	*	A B +
7	(	push	* (	A B +
8	C	output	* (	A B + C

Step	Token	Action	Stack	Output
9	-	push (top is `(`)	* (-	A B + C
10	D	output	* (-	A B + C D
11	)	pop until `(`: pop - → output; pop (→ discard	*	A B + C D -
End	—	pop remaining: *	(empty)	A B + C D - *

Final Postfix: `A B + C D - *`

5.5 Time and Space Complexity

- Time: **O(n)** — each token is pushed and popped from the stack at most once.
- Space: **O(n)** — worst case (e.g., all operators of increasing precedence, or deeply nested parentheses) the stack holds up to n elements.

---

6. Converting Infix to Prefix

There is no direct "shunting yard for prefix" that scans left to right the same way — instead, the cleanest method is a clever 3-step trick:

6.1 The Algorithm

1. **Reverse** the infix expression. While reversing, swap every `(` with `)` and every `)` with `(`.
2. **Convert this reversed expression to postfix** using the exact infix-to-postfix algorithm from Section 5 — but with **one modification**: when comparing precedence for equal-precedence operators, treat them as if scanning is happening on the reversed string, so equal-precedence operators here should NOT be popped (i.e., use a "greater than or equal to" pop condition flipped to strictly "greater than" for left-associative operators, to preserve original left-to-right order after reversal). In most textbook implementations this is simplified as: use the same algorithm, but if two operators have equal precedence, do **not** pop (push instead) — this correctly reconstructs original ordering upon final reversal.
3. **Reverse the resulting postfix string** to get the prefix expression.

6.2 Worked Example

Convert `A + B * C` to prefix.

Step 1 — Reverse and swap brackets: Original: `A + B * C` Reversed: `C * B + A` (no brackets here, so no swapping needed)

Step 2 — Convert `C * B + A` to postfix (using modified rule: equal precedence → don't pop):

Token	Action	Stack	Output
C	output	—	C
*	push	*	C
B	output	*	C B
+	prec(+) < prec*() on stack* → *pop* to output; stack empty → push +	+	C B *
A	output	+	C B * A
End	pop remaining: +	—	C B * A +

Intermediate postfix (of reversed string): `C B * A +`

Step 3 — Reverse this result: `+ A * B C`

Final Prefix: `+ A * B C`

Sanity check by reading prefix right-to-left with a stack (Section 3.2): push C, push B, see `*` → pop B,C → compute B_C → push (B_C); push A; see `+` → pop A,(B_C) → compute A+(B_C) → push (A+(B_C)). Result = A + (B_C), which matches original infix precedence (multiplication before addition). .

---

7. Converting Postfix to Infix

7.1 Algorithm

Scan postfix **left to right** using a stack:

1. If token is an **operand**, push it onto the stack as-is.
2. If token is an **operator**, pop the top two elements — call them `op2` (popped first) then `op1` (popped second) — form the string `(op1 <operator> op2)` and push this new string back onto the stack.
3. At the end, the single element remaining on the stack is the fully parenthesized infix expression.

Important: the order matters — the *first* popped element is the *right-hand* operand, and the *second* popped element is the *left-hand* operand, because in postfix the operands appear in original left-to-right order, and a stack reverses the pop order.

7.2 Worked Example

Convert postfix `A B C * + D -` to infix.

Token	Action	Stack
A	push	A
B	push	A, B
C	push	A, B, C
*	pop C(op2), B(op1) → form `(B*C)` → push	A, (B*C)
+	pop (B*C)(op2), A(op1) → form `(A+(B*C))` → push	(A+(B*C))
D	push	(A+(B*C)), D
-	pop D(op2), (A+(B*C))(op1) → form `((A+(B*C))-D)` → push	((A+(B*C))-D)

Final Infix: `((A + (B * C)) - D)` — matches the original expression from Section 5.3.

---

8. Converting Prefix to Infix

8.1 Algorithm

Scan prefix **right to left** using a stack:

1. If token is an **operand**, push it.
2. If token is an **operator**, pop the top two elements — call the first popped `op1` and the second popped `op2` — form `(op1 <operator> op2)` and push it back.
   - Note: here, since we scan right-to-left, the *first* popped element is the *left-hand* operand (it appeared first, i.e., more to the left, in the original left-to-right reading) — this is the mirror image of the postfix case.
3. The final remaining stack element is the infix expression.

8.2 Worked Example

Convert prefix `+ A * B C` to infix.

Scan right to left: `C`, `B`, `*`, `A`, `+`

Token	Action	Stack
C	push	C
B	push	C, B
*	pop B(op1), C(op2) → form `(B*C)` → push	(B*C)

Token	Action	Stack
A	push	(B*C), A
+	pop A(op1), (B*C)(op2) → form `(A+(B*C))` → push	(A+(B*C))

Final Infix: `(A + (B * C))` . matches Section 6.2.

---

9. Converting Postfix to Prefix (and vice versa) — Direct Methods

9.1 Postfix → Prefix (Direct, No Detour Through Infix)

Scan postfix **left to right**:

1. Operand → push onto stack.
2. Operator → pop top two elements, `op2` then `op1` (same order as Section 7.1). Form the **prefix-style** string: `<operator> op1 op2` (operator first now, not in the middle). Push this back.
3. Final stack element is the prefix expression.

Example: Postfix `A B C * +` →

- push A, push B, push C
- `*` : pop C(op2), B(op1) → form `* B C` → push
- `+` : pop `* B C` (op2), A(op1) → form `+ A * B C` → push

Result: `+ A * B C` (matches Section 6's answer).

9.2 Prefix → Postfix (Direct)

Scan prefix **right to left**:

1. Operand → push.
2. Operator → pop top two, `op1` then `op2` (mirrors Section 8.1 order). Form **postfix-style** string: `op1 op2 <operator>` . Push back.
3. Final stack element is the postfix expression.

Example: Prefix `+ A * B C` → scan right to left: C, B, *, A, +

- push C, push B
- `*` : pop B(op1), C(op2) → form `B C *` → push

- push A
- `+`: pop A(op1), `B C *`(op2) → form `A B C * +` → push

Result: `A B C * +` . matches Section 5.3.

---

10. Evaluating Postfix Expressions

This is the algorithm you will implement most often in interviews and DSA assignments.

10.1 Algorithm

1. Initialize an empty stack.
2. Scan the postfix expression **left to right**, token by token.
3. If the token is an **operand** (number), push its numeric value onto the stack.
4. If the token is an **operator**: a. Pop the top element → call it `b` (the second operand, since it appeared later but is popped first). b. Pop the next top element → call it `a` (the first operand). c. Compute `result = a <operator> b` (note the order: `a` first, `b` second — this matters critically for non-commutative operators like `-` and `/`). d. Push `result` back onto the stack.
5. After the scan finishes, the stack contains exactly **one** element — that is the final answer. (If more than one element remains, the expression was malformed.)

10.2 Worked Example

Evaluate postfix `6 2 3 + - 3 8 2 / + *`

Token	Action	Stack
6	push	6
2	push	6, 2
3	push	6, 2, 3
+	pop 3(b), 2(a) → 2+3=5 → push	6, 5
-	pop 5(b), 6(a) → 6-5=1 → push	1
3	push	1, 3
8	push	1, 3, 8
2	push	1, 3, 8, 2
/	pop 2(b), 8(a) → 8/2=4 → push	1, 3, 4
+	pop 4(b), 3(a) → 3+4=7 → push	1, 7

Token	Action	Stack
*	pop 7(b), 1(a) → 1*7=7 → push	7

Final Answer: 7

10.3 Complexity

- Time: **O(n)** — single pass, constant-time push/pop/arithmetic per token.
- Space: **O(n)** — worst case, all operands pushed before any operator appears (e.g., `A B C D +++` -like dense operand runs).

---

11. Evaluating Prefix Expressions

11.1 Algorithm

1. Scan the prefix expression **right to left**.
2. If token is an operand, push it.
3. If token is an operator: pop top element → `a` (first popped), pop next → `b` (second popped). Compute `result = a <operator> b` (note: here `a`, the first popped, is the **left** operand since we're scanning right-to-left — this is the mirror image of postfix's order). Push result.
4. Final remaining stack element is the answer.

11.2 Worked Example

Evaluate prefix `* + 2 3 4` (equivalent to infix `(2+3)*4`)

Scan right to left: `4`, `3`, `2`, `+`, `*`

Token	Action	Stack
4	push	4
3	push	4, 3
2	push	4, 3, 2
+	pop 2(a), 3(b) → 2+3=5 → push	4, 5
*	pop 5(a), 4(b) → 5*4=20 → push	20

Final Answer: 20 . matches (2+3)*4 = 20.

12. Building an Expression Tree (The Unified Underlying Structure)

All three notations are just different **traversals of the same binary expression tree**, where every leaf is an operand and every internal node is an operator with exactly two children (for binary operators).

- **Preorder traversal** (Root → Left → Right) of the expression tree = **Prefix**
- **Inorder traversal** (Left → Root → Right) of the expression tree = **Infix** (add parentheses around each subtree to preserve grouping)
- **Postorder traversal** (Left → Right → Root) of the expression tree = **Postfix**

12.1 Building a Tree from Postfix (Most Common Interview Question)

Algorithm: scan postfix left to right with a **stack of tree nodes**.

1. Operand → create a leaf node, push onto stack.
2. Operator → pop two nodes, `right` (popped first) and `left` (popped second). Create a new node with this operator, `left` as its left child, `right` as its right child. Push this new node.
3. At the end, the single node remaining is the **root** of the expression tree.

This is the same "two-pop" logic as postfix evaluation (Section 10), except instead of computing a numeric result you build a node.

12.2 Building a Tree from Prefix

Same idea but scan **right to left**, and when popping two nodes for an operator, the first popped becomes the **left** child, the second popped becomes the **right** child (mirroring Section 11's argument order).

12.3 Why This Matters

Once you have the expression tree, you can:

- Evaluate it recursively (evaluate left subtree, evaluate right subtree, apply operator).
- Regenerate any of the three notations via the appropriate traversal.
- This is the conceptual bridge between "expression notation problems" and "binary tree problems" — a very common way interview questions combine two DSA topics into one.

13. Common Pitfalls (Read This Before Coding)

1. **Order of pop matters.** In postfix evaluation, the first popped value is the *right* operand (`b`), the second popped is the *left* operand (`a`). Getting this backwards silently breaks every non-commutative operator (`-` , `/` , `^`) while commutative ones (`+` , `*`) will still "work" and hide the bug.
2. **Right-associativity of `^` is the #1 source of bugs** in infix-to-postfix conversion. If you treat `^` like `*` or `/` (pop on equal precedence), `A ^ B ^ C` will incorrectly convert to a form that evaluates as `(A^B)^C` instead of the mathematically standard `A^(B^C)` .
3. **Single-character vs. multi-character operands.** All examples above assume single-letter/digit operands for simplicity. Real implementations must tokenize multi-digit numbers or multi-character identifiers properly (e.g., using a tokenizer/lexer that reads full numbers, not one digit at a time) before applying any of these algorithms.
4. **Mismatched parentheses.** If, after scanning, `)` is encountered but no `(` is found on the stack (or at the end, a `(` still remains unmatched on the stack), the expression is invalid — always add this validation in production code.
5. **Empty stack pop.** In evaluation, always check the stack has at least two elements before popping for a binary operator — a malformed expression will otherwise crash your program.
6. **Unary operators (like unary minus `-5`)** are not covered by the binary algorithms above and need special-case handling (commonly by inserting a dummy `0` before the unary minus, or tracking whether an operator is in "unary position" based on the previous token).

---

14. Summary Cheat-Sheet

Task	Scan Direction	Key Data Structure	Pop Order Meaning
Infix → Postfix	Left to Right	1 stack (operators)	precedence + associativity rules
Infix → Prefix	Reverse + convert + reverse	1 stack (operators)	equal precedence → don't pop
Postfix → Infix	Left to Right	1 stack (strings)	1st pop = right operand
Prefix → Infix	Right to Left	1 stack (strings)	1st pop = left operand
Postfix → Prefix	Left to Right	1 stack (strings)	1st pop = right operand
Prefix → Postfix	Right to Left	1 stack (strings)	1st pop = left operand
Evaluate Postfix	Left to Right	1 stack (numbers)	1st pop = b (right), 2nd pop = a (left); compute a op b

Task	Scan Direction	Key Data Structure	Pop Order Meaning
Evaluate Prefix	Right to Left	1 stack (numbers)	1st pop = a (left), 2nd pop = b (right); compute a op b
Build tree from Postfix	Left to Right	1 stack (nodes)	1st pop = right child, 2nd pop = left child
Build tree from Prefix	Right to Left	1 stack (nodes)	1st pop = left child, 2nd pop = right child

All conversions and evaluations run in **O(n) time and O(n) space**.

---

15. Practice Questions (Increasing Difficulty)

Q1 (Very Basic). Convert the infix expression `A + B` to both postfix and prefix notation.

Q2 (Basic). Evaluate the postfix expression `5 1 2 + 4 * + 3 -` by hand, showing the stack contents after every token.

Q3 (Medium). Convert the infix expression `A + B * C - (D / E ^ F)` to postfix notation, showing the full stack/output trace table (like Section 5.3), being careful with the right-associativity of `^`.

Q4 (Hard). Given the postfix expression `A B + C D - * E /`, (a) draw the corresponding binary expression tree, (b) derive the equivalent prefix expression directly from the tree (preorder traversal), and (c) derive the original infix expression from the tree (inorder traversal with parentheses).

Q5 (Very Hard). Write a single function `convert(expr, from_notation, to_notation)` (in any language of your choice) that supports all six conversions listed in the Section 14 cheat-sheet using ONE shared internal representation (hint: always build the expression tree first, then traverse it appropriately for the requested output notation). Your function must correctly handle: multi-digit operands, right-associative `^`, and nested parentheses of arbitrary depth. Additionally, prove the time complexity of your unified approach is still O(n) regardless of which of the six conversions is requested.